Smart Hostel Management System

Database Project Report

GROUP MEMBERS:

SAHIL KUMAR : (24K-0941)

SHAMVEEL KHAN : (24K-0962)

KABEER JAWED : (24K-0700)

INSTRUCTOR : SIR SHOHAIB RAUF

1. DESIGN PHASE

a) Proposal

i. Introduction and Brief Description of the System

The **Smart Hostel Management System** is a web-based application designed to efficiently manage hostel operations such as student registration, room allocation, fee management, complaints, and visitor tracking.

Hostels in universities often rely on manual records or poorly organized spreadsheets, which can cause problems such as room overbooking, difficulty tracking payments, and inefficient complaint handling.

The proposed system will provide a centralized database that allows hostel administrators to manage student information, room allocation, payments, and complaints in an organized and automated way. It will reduce manual errors, improve efficiency, and provide quick access to hostel records.

The system will allow authorized users to perform operations such as adding students, assigning rooms, tracking hostel fees, and managing complaints through an easy-to-use web interface.

ii. System Functionality

The system will perform the following functions:

1. Student Management

- Register new hostel students
- Store student personal information
- View and update student records

2. Room Management

- Store room information
- Track room capacity
- Assign students to rooms
- Prevent room overbooking

3. Hostel Fee Management

- Record hostel fee payments
- Track unpaid students
- Generate fee reports

4. Complaint Management

- Students can submit complaints
- Staff can view and resolve complaints
- Complaint status tracking (Pending / Resolved)

5. Visitor Management

- Record hostel visitors
- Track visitor entry and exit time

6. Reporting System

The system will generate reports such as:

- Room occupancy report
- Unpaid fee report
- Complaint status report

---

iii. Users of the System

The system will have three main user roles:

1. Administrator

The administrator manages the entire hostel system.

Responsibilities:

- Add or remove student records
- Manage room information

- Allocate rooms to students
- Manage hostel staff
- View all system reports

---

2. Hostel Staff

Hostel staff handle daily hostel operations.

Responsibilities:

- Record student fee payments
- Manage complaints
- Track visitor entry
- Update student information

---

3. Students

Students interact with the system for limited operations.

Responsibilities:

- Submit complaints
- View room information
- View payment history

---

iv. Technologies Used

The system will be implemented as a **Web Application** using the following technologies:

Frontend

- HTML
- CSS
- JavaScript

Backend

- Python with Flask

Database

- MySQL (SQL based database)

The database will store all hostel information such as students, rooms, payments, and complaints.

---

v. Scope of the System

What the System Will Cover

The system will manage:

- Student hostel records
- Room allocation
- Hostel fee tracking
- Complaint management
- Visitor records
- Basic reporting

What the System Will NOT Cover

The system will not include:

- Online payment gateways
- Mobile applications
- Biometric attendance systems
- External hostel security integration

The project focuses mainly on **database management and web-based hostel administration.**

---

vi. Assumptions

The following assumptions are made during system design:

- Each student is assigned to only one room.
- Each room has a fixed capacity.
- Only registered students can submit complaints.
- Hostel staff are responsible for managing complaints and visitor records.
- Administrators have full access to the system.

---

b) Normalization

The normalization process is applied to ensure the database is efficient and free from redundancy.

Step 1: Unnormalized Form (UNF)

Initial data may look like this:

StudentID StudentName RoomID RoomCapacity FeeAmount ComplaintID ComplaintType

Problems:

- Data redundancy
- Multiple complaints per student
- Repeating groups

Step 2: First Normal Form (1NF)

Conditions:

- Remove repeating groups
- Each field contains atomic values

Example:

Student Table

StudentID Name Email Phone

Complaint Table

| ComplaintID | StudentID | ComplaintType | Status |

Step 3: Second Normal Form (2NF)

Conditions:

- Remove partial dependency
- Non-key attributes must depend on the entire primary key

Example tables:

Students(StudentID, Name, Phone)

Rooms(RoomID, Capacity)

Payments(PaymentID, StudentID, Amount)

Step 4: Third Normal Form (3NF)

Conditions:

- Remove transitive dependencies

Example:

Students(StudentID, Name)

Rooms(RoomID, Capacity)

Allocations(StudentID, RoomID)

Payments(PaymentID, StudentID, Amount)

Complaints(ComplaintID, StudentID, Description)

Step 5: Boyce-Codd Normal Form (BCNF)

BCNF Rule:
Every determinant must be a candidate key.

All tables satisfy BCNF because:

- Each determinant uniquely identifies the record.

---

c) Entity Relationship Diagram (ERD)

Entities Used in the System

1. Student

Stores information about hostel students.

Attributes:

- StudentID (Primary Key)
- Name
- Email
- Phone
- Department

---

2. Room

Stores hostel room details.

Attributes:

- RoomID (Primary Key)
- RoomNumber
- Capacity

---

3. RoomAllocation

Links students with rooms.

Attributes:

- AllocationID (Primary Key)
- StudentID (Foreign Key)
- RoomID (Foreign Key)

---

4. Payment

Stores hostel fee payments.

Attributes:

- PaymentID (Primary Key)
- StudentID (Foreign Key)
- Amount
- PaymentDate

---

5. Complaint

Stores student complaints.

Attributes:

- ComplaintID (Primary Key)
- StudentID (Foreign Key)
- Description
- Status

---

6. Visitor

Stores visitor information.

Attributes:

- VisitorID (Primary Key)
- StudentID (Foreign Key)
- VisitorName
- EntryTime
- ExitTime

ER Relationships

Student → RoomAllocation → Room
Student → Payment

Student → Complaint
Student → Visitor

---

2. IMPLEMENTATION PHASE

Tables

Example tables implemented in SQL:

Students Table

```
CREATE TABLE Students (
StudentID INT PRIMARY KEY,
Name VARCHAR(50),
Email VARCHAR(50),
Phone VARCHAR(15),
Department VARCHAR(50)
);
```

Rooms Table

```
CREATE TABLE Rooms (
RoomID INT PRIMARY KEY,
RoomNumber INT,`
Capacity INT
);
```

Payments Table

```
CREATE TABLE Payments (
PaymentID INT PRIMARY KEY,
StudentID INT,
Amount DECIMAL(10,2),
PaymentDate DATE,
FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Views

Example views:

Unpaid Students View

```
CREATE VIEW UnpaidStudents AS
SELECT Students.Name
FROM Students
LEFT JOIN Payments
ON Students.StudentID = Payments.StudentID
WHERE Payments.StudentID IS NULL;
```

Example Queries

Join Query

```
SELECT Students.Name, Rooms.RoomNumber
FROM Students
JOIN RoomAllocation
ON Students.StudentID = RoomAllocation.StudentID
JOIN Rooms
ON RoomAllocation.RoomID = Rooms.RoomID;
```

Built-in SQL Functions

Example:

```
SELECT COUNT(*) AS TotalStudents
FROM Students;
```

Insert Operation

```
INSERT INTO Students
VALUES (101,'Ali Khan','ali@email.com','123456789','CS');
```

Update Operation

```
UPDATE Students
SET Phone = '987654321'
WHERE StudentID = 101;
```

Delete Operation

```
DELETE FROM Students
WHERE StudentID = 101;
```

Transaction Handling

Example:

```
START TRANSACTION;

INSERT INTO Payments VALUES (1,101,5000,'2026-05-01');

COMMIT;
```

Programs for Each User Role

Administrator

- Manage students
- Allocate rooms
- Generate reports

Hostel Staff

- Record payments
- Manage complaints
- Record visitor entries

Students

- Submit complaints
- View room information
- View payment history

✅ This report satisfies:

✔ Proposal
✔ Normalization up to BCNF
✔ 5–6 entities
✔ 8–9 tables possible
✔ SQL implementation

- ✔ Queries
- ✔ Transactions
- ✔ User roles